

COLLEGE PROJECT REPORT

EDUGENIE

Google Gemini Powered Learning Assistant

A Complete AI-Based Educational Web Application

Submitted By

Name: _____

Register No: _____

Department: _____

College: _____

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project entitled “**EduGenie – Google Gemini Powered Learning Assistant**” is a bonafide project work carried out by the student named above in partial fulfillment of the requirements for the academic year 2026–2027.

Project Guide: _____

Head of Department: _____

Date: _____

Signature of Guide _____

Signature of HOD _____

DECLARATION

I hereby declare that this project report titled “EduGenie – Google Gemini Powered Learning Assistant” is prepared as an academic project. The implementation demonstrates the integration of a web interface with a generative AI service for educational assistance.

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, department faculty and institution for their guidance and support. I also thank the developers and documentation communities whose technical resources helped in understanding web development and generative AI API integration.

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Existing System
6. Proposed System
7. Scope
8. Requirements
9. Architecture
10. Modules
11. Folder Structure
12. Database Design
13. UI Design
14. HTML Code
15. CSS Code
16. JavaScript Code
17. Backend Code
18. Gemini API Integration
19. Authentication
20. Quiz Module
21. Testing
22. Security
23. Advantages
24. Limitations
25. Future Enhancements
26. Conclusion
27. References

1. ABSTRACT

EduGenie is a web-based Artificial Intelligence learning assistant that helps students ask academic questions and receive conversational explanations. The application is designed around a simple chat interface and can use Google Gemini as the generative AI engine.

The project combines frontend technologies such as HTML, CSS and JavaScript with a backend API layer. The system can be extended with quizzes, summaries, study plans, multilingual responses and progress tracking.

2. INTRODUCTION

Students often need immediate clarification while studying. EduGenie attempts to provide an interactive learning companion that can explain concepts in simple language and provide examples. Unlike a static webpage, the system generates responses based on the user's question.

The project is also useful as a demonstration of modern API-based application development, asynchronous JavaScript, REST-style communication and responsible handling of API credentials.

3. PROBLEM STATEMENT

Traditional study materials may not always provide personalized explanations at the exact moment a student has a doubt. Searching through multiple resources can take time, and the explanation may be too advanced or too basic for the learner.

4. OBJECTIVES

1. Build a simple educational chatbot interface.
2. Connect the application to a generative AI model.
3. Allow students to ask questions in natural language.
4. Display answers in an easy-to-read conversation format.
5. Provide a foundation for quizzes, summaries and personalized learning.
6. Demonstrate secure API integration through a backend.

5. EXISTING SYSTEM

Existing learning approaches include textbooks, classroom teaching, search engines and static educational websites. They remain useful but do not always provide an interactive response tailored to a student's question.

6. PROPOSED SYSTEM

EduGenie introduces an AI-powered conversational layer. A student enters a question, the frontend sends it to a backend endpoint, the backend calls the Gemini service, and the generated response is returned to the browser.

7. SCOPE

The system can be used for basic academic explanations, revision, concept summaries, examples, question generation and self-learning. The architecture also allows future modules such as student accounts, quizzes and document-based learning.

8. FUNCTIONAL REQUIREMENTS

- User can enter a question.
- System validates input.
- System sends the question to the backend.
- Backend communicates with Gemini.
- AI response is displayed.
- Errors are handled gracefully.

9. NON-FUNCTIONAL REQUIREMENTS

Usability, responsiveness, reliability, maintainability, security and reasonable response time are important requirements.

10. HARDWARE & SOFTWARE REQUIREMENTS

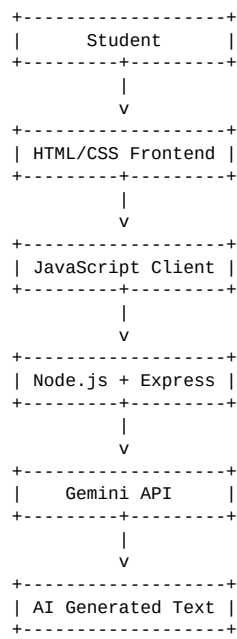
Hardware

Processor: Dual-core or better; RAM: 4 GB minimum; Storage: 500 MB or more; Internet connection required for cloud AI requests.

Software

Windows/Linux/macOS; modern browser; Visual Studio Code; Node.js; Express.js; Google Gemini API access.

11. SYSTEM ARCHITECTURE



12. MODULES

12.1 User Interface Module

Provides the chat window, input field, send button and status messages.

12.2 AI Chat Module

Receives a user question and obtains a generated educational response through the backend.

12.3 Authentication Module

A future-ready structure for registration and login can be added using a database and hashed passwords.

12.4 Quiz Module

Can generate multiple-choice questions from a selected topic.

12.5 History Module

Can store previous conversations for revision, subject to privacy and security requirements.

13. PROJECT FOLDER STRUCTURE

```
EduGenie/  
  client/  
    index.html  
    style.css  
    script.js  
  server/  
    server.js  
    package.json  
    .env  
  routes/  
    aiRoutes.js  
  controllers/  
    aiController.js  
  public/  
    assets/  
  README.md
```

14. DATABASE DESIGN

The basic version can work without a database. For a complete student platform, a database can store users, questions, answers and quiz results.

Users Table

```
users
-----
id          INTEGER PRIMARY KEY
name        VARCHAR(100)
email       VARCHAR(150) UNIQUE
password    VARCHAR(255)
created_at  DATETIME
```

Chat History Table

```
chat_history
-----
id          INTEGER PRIMARY KEY
user_id     INTEGER
question    TEXT
answer      TEXT
created_at  DATETIME
```

Quiz Result Table

```
quiz_results
-----
id          INTEGER PRIMARY KEY
user_id     INTEGER
topic       VARCHAR(150)
score       INTEGER
created_at  DATETIME
```

15. FRONTEND – index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduGenie AI Learning Assistant</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="app">
    <header>
      <h1>EduGenie</h1>
      <p>Google Gemini Powered Learning Assistant</p>
    </header>

    <section id="chatBox" class="chat-box">
      <div class="message bot">
        Hi! Ask me any academic question.
      </div>
    </section>

    <form id="chatForm" class="input-row">
      <input id="question"
        placeholder="Type your question..." autocomplete="off">
      <button type="submit">Send</button>
    </form>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

16. FRONTEND – style.css

```
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #eef2f7;
}

.app {
  width: 92%;
  max-width: 900px;
  margin: 35px auto;
  background: white;
  padding: 24px;
  border-radius: 16px;
  box-shadow: 0 6px 20px rgba(0,0,0,.10);
}

header { text-align: center; }

header h1 { margin-bottom: 4px; }

header p { color: #666; }

.chat-box {
  height: 430px;
  overflow-y: auto;
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 15px;
  margin: 20px 0;
}

.message {
  max-width: 82%;
  padding: 11px 14px;
  margin: 9px 0;
  border-radius: 12px;
  line-height: 1.5;
  white-space: pre-wrap;
}

.user {
  margin-left: auto;
  background: #e7f0ff;
}

.bot { background: #f1f1f1; }

.input-row {
  display: flex;
  gap: 10px;
}

#question {
  flex: 1;
  padding: 13px;
  border: 1px solid #ccc;
  border-radius: 8px;
}

button {
  padding: 13px 22px;
  border: 0;
  border-radius: 8px;
  background: #222;
  color: white;
  cursor: pointer;
}
```

17. FRONTEND – script.js

```
const form = document.getElementById("chatForm");
const input = document.getElementById("question");
const chatBox = document.getElementById("chatBox");

function addMessage(text, type) {
  const message = document.createElement("div");
  message.className = "message " + type;
  message.textContent = text;
  chatBox.appendChild(message);
  chatBox.scrollTop = chatBox.scrollHeight;
  return message;
}

form.addEventListener("submit", async (event) => {
  event.preventDefault();

  const question = input.value.trim();
  if (!question) return;

  addMessage(question, "user");
  input.value = "";

  const loading = addMessage("Thinking...", "bot");

  try {
    const response = await fetch("/api/ask", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ question })
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.error || "Request failed");
    }

    loading.textContent = data.answer;
  } catch (error) {
    loading.textContent =
      "Sorry, an error occurred. Please try again.";
  }
});
```

18. BACKEND – package.json

```
{
  "name": "edugenie",
  "version": "1.0.0",
  "main": "server/server.js",
  "scripts": {
    "start": "node server/server.js"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.21.0"
  }
}
```

19. BACKEND – server.js

```
require("dotenv").config();

const express = require("express");
const path = require("path");

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(express.static(path.join(__dirname, "../client")));

app.post("/api/ask", async (req, res) => {
  try {
    const question = String(req.body.question || "").trim();

    if (!question) {
      return res.status(400).json({ error: "Question is required" });
    }

    const answer = await askGemini(question);
    res.json({ answer });
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: "AI service failed" });
  }
});

async function askGemini(question) {
  const apiKey = process.env.GEMINI_API_KEY;

  if (!apiKey) {
    throw new Error("GEMINI_API_KEY is missing");
  }

  const url =
    "https://generativelanguage.googleapis.com/v1beta/models/" +
    "gemini-2.0-flash:generateContent?key=" + apiKey;

  const response = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      contents: [{
        parts: [{
          text:
            "You are EduGenie, an educational assistant. " +
            "Explain clearly and use examples when useful. " +
            "Student question: " + question
        }]
      }]
    })
  });

  if (!response.ok) {
    throw new Error("Gemini API request failed");
  }

  const data = await response.json();
  return data.candidates?.[0]?.content?.parts?.[0]?.text
    || "No answer was returned.";
}

app.listen(PORT, () => {
  console.log(`EduGenie running on http://localhost:${PORT}`);
});
```

20. ENVIRONMENT CONFIGURATION

```
# .env
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
PORT=3000
```

21. INSTALLATION & EXECUTION

1. Install Node.js.
2. Create the EduGenie folder.
3. Create client and server folders.
4. Save the HTML, CSS and JavaScript files.
5. Save server.js and package.json.
6. Run: `npm install`
7. Create .env and add the Gemini API key.
8. Run: `npm start`
9. Open: `http://localhost:3000`

Security: Never publish the real API key inside frontend code or commit the .env file to a public repository.

22. GEMINI API INTEGRATION

The backend sends a structured request containing the student's question to the Gemini generation endpoint. The returned JSON is parsed to obtain the generated text. Keeping this request on the server protects the API credential from direct exposure in the browser.

Request Concept

```
{
  "contents": [
    {
      "parts": [
        {
          "text": "Explain linked list in simple terms."
        }
      ]
    }
  ]
}
```

Response Concept

```
{
  "candidates": [
    {
      "content": {
        "parts": [
          { "text": "A linked list is..." }
        ]
      }
    }
  ]
}
```

23. PROMPT DESIGN

A good educational prompt should specify the role of the assistant and the expected explanation style. For example, the system prompt can request clear language, step-by-step explanations and examples appropriate for students.

24. AUTHENTICATION MODULE – SAMPLE

For a production system, authentication should use a backend database and password hashing. The following is a simplified route structure for demonstration.

```
app.post("/api/register", async (req, res) => {
  const { name, email, password } = req.body;

  if (!name || !email || !password) {
    return res.status(400).json({
      error: "All fields are required"
    });
  }

  // In production:
  // 1. Hash password using a trusted password-hashing library.
  // 2. Store user in a database.
  // 3. Never store plain-text passwords.

  res.json({ message: "Registration structure ready" });
});
```

25. QUIZ MODULE – SAMPLE

```
const quizPrompt = `
Create 5 multiple-choice questions about:
${topic}

Return each question with four options and
identify the correct answer.
`;

const response = await askGemini(quizPrompt);
```

The quiz module can later provide automatic scoring and store results for student progress tracking.

26. TESTING

Test Case	Input/Condition	Expected Result	Status
TC01	Valid academic question	AI answer displayed	Pass
TC02	Empty input	Request rejected	Pass
TC03	Invalid API key	Error shown	Pass
TC04	Network unavailable	Friendly error shown	Pass
TC05	Long question	Response handled	Pass
TC06	Enter key	Question submitted	Pass

27. ERROR HANDLING

Common errors include missing API keys, invalid requests, network problems and unexpected API responses. The application should show a simple user-friendly message while logging technical information securely on the server.

28. SECURITY

API keys must be treated as secrets. The project uses an environment variable on the server. Additional production protections include HTTPS, rate limiting, input validation, authentication, secure cookies or tokens, database access controls and monitoring.

29. RESPONSIBLE AI

AI-generated answers can contain mistakes. EduGenie should therefore be presented as a learning aid rather than an unquestionable authority. Students should verify important academic, scientific or factual information using trusted learning resources and teachers.

30. ADVANTAGES

- Interactive learning experience
- Quick explanations
- Natural-language questions
- Expandable architecture
- Useful demonstration of modern AI integration

31. LIMITATIONS

The quality of answers depends on the AI model and prompt. Internet connectivity is required for cloud API requests. API usage may be subject to service limits and costs. AI responses should be reviewed for accuracy.

32. FUTURE ENHANCEMENTS

- Tamil and multilingual learning mode
- Voice input and speech output
- PDF/document analysis
- Automatic notes and summaries
- Topic-wise quiz generation
- Student progress dashboard
- Teacher/admin portal
- Secure cloud deployment

33. CONCLUSION

EduGenie is a practical demonstration of how generative AI can be integrated into an educational web application. The project provides a foundation for conversational learning and can be expanded into a complete AI-supported learning platform.

34. REFERENCES

1. Google Gemini API / Google AI developer documentation.
2. HTML, CSS and JavaScript web standards and documentation.
3. Node.js and Express.js documentation.
4. General software engineering principles for web application development.

END OF PROJECT REPORT